CUỘC THI SINH VIÊN SOICT 2026

Vận Tải Tích Hợp Hành Khách và Bưu Kiện
(Integrated Passenger and Parcel Transportation)

Tác giả đề bài:

TS. Phạm Quang Dũng, TS. Dương Quang Huy, TS. Nguyễn Hùng Sơn

Người thực hiện: Bùi Thế Dũng, Nguyễn Hà Thu (UET-VNU)

Ngày 11 tháng 9 năm 2026

Mục lục

1 Bản dịch chính thức đề bài **2**
1.1 Phát biểu Bài toán . . . 2
1.2 Yêu cầu Hành khách (Passenger Requests) . . . 2
1.3 Yêu cầu Bưu kiện (Parcel Requests) . . . 3
1.4 Sức chứa của Taxi (Taxi Capacity) . . . 3
1.5 Lộ trình và Chi phí Di chuyển (Routes and Travel Cost) . . . 3
1.6 Hàm Mục tiêu (Objective Function) . . . 4

2 Phân tích yêu cầu bài toán & Bảng tóm tắt **5**
2.1 Tóm tắt Đọc hiểu Bài toán . . . 5
2.2 Bảng Tóm tắt Ràng buộc Cứng (Hard Constraints) . . . 5
2.3 Danh mục Các Biến Dữ Liệu Cần Quản Lý Trong Python . . . 5

3 Kế hoạch & Chiến thuật **6**
3.1 Định Hướng Chiến Thuật Thuật Toán . . . 6
3.2 Mốc Thời Gian Quan Trọng (Timeline Key Dates) . . . 6
3.3 Lộ Trình Chi Tiết Từng Tuần (Giai Đoạn Probation) . . . 6
3.3.1 Tuần 1 (20/09 – 27/09): Nền Tảng Data Structure & Lời Giải Cơ Sở . . . 6
3.3.2 Tuần 2 (28/09 – 04/10): Tối Ưu Cục Bộ (Local Search) . . . 6
3.3.3 Tuần 3 (05/10 – 12/10): Tích Hợp ALNS . . . 7
3.3.4 Tuần 4 (13/10 – 20/10): Tinh Chỉnh Tham Số & Tối Ưu Tốc Độ . . . 7
3.4 Giai Đoạn Vòng Chung Kết (01/11/2026 – 04/11/2026) . . . 7

1 Bản Dịch Chính Thức Đề Bài (Problem Statement)

1.1 Phát biểu Bài toán

Một công ty vận tải có K xe taxi, được đánh số $k \in \{1, \ldots, K\}$. Taxi k ban đầu đặt tại điểm O_k và sẵn sàng phục vụ từ thời điểm 0. Một taxi có thể kết thúc hành trình ở bất kỳ điểm nào và không bắt buộc phải quay lại vị trí xuất phát ban đầu.

Công ty nhận được N yêu cầu chở hành khách và M yêu cầu vận chuyển bưu kiện. Một yêu cầu phải được phục vụ hoàn toàn hoặc không được phục vụ; không chấp nhận phục vụ một phần. Mỗi yêu cầu được phục vụ sẽ do đúng một xe taxi đảm nhận. Các xe taxi phải thỏa mãn tất cả các ràng buộc về khung thời gian và sức chứa bưu kiện.

Thời gian di chuyển và chi phí di chuyển từ điểm u đến điểm v lần lượt được ký hiệu là $t(u, v)$ và $c(u, v)$. Các giá trị này được cho trước đối với mọi cặp điểm có hướng. Thời gian và chi phí di chuyển luôn không âm, và không nhất thiết phải có tính đối xứng ($t(u, v) \neq t(v, u)$ hoặc $c(u, v) \neq c(v, u)$).

1.2 Yêu cầu Hành khách (Passenger Requests)

Yêu cầu hành khách i ($1 \leq i \leq N$) được xác định bởi:

- Điểm đón: P_i^n;
- Điểm trả: D_i^n;
- Doanh thu: T_i^n;
- Khung thời gian đón: $[E_i^n, L_i^n]$;
- Thời gian dịch vụ đón và trả: S_i^n.

Nếu taxi k phục vụ yêu cầu khách i, gọi $arrP_i^n$ là thời điểm xe đến điểm đón P_i^n. Việc đón khách chỉ hợp lệ khi:

$$arrP_i^n \leq L_i^n \tag{1}$$

Thời gian bắt đầu dịch vụ đón khách là:

$$srvP_i^n = \max\{arrP_i^n, E_i^n\} \tag{2}$$

và hoàn thành lúc:

$$depP_i^n = srvP_i^n + S_i^n \tag{3}$$

Sau khi đón khách, taxi phải di chuyển trực tiếp đến D_i^n. Trong khoảng thời gian giữa đón và trả khách i, taxi **không được phép** ghé thăm hoặc phục vụ bất kỳ điểm đón/trả nào thuộc về yêu cầu khác. Đặc biệt, **không bưu kiện nào được phép lấy hoặc giao trong khoảng thời gian này**. Do đó:

$$arrD_i^n = depP_i^n + t(P_i^n, D_i^n) \tag{4}$$

Dịch vụ trả khách bắt đầu ngay lập tức khi đến nơi và hoàn thành lúc:

$$depD_i^n = arrD_i^n + S_i^n \tag{5}$$

**Ghi chú: Một taxi chỉ có thể chở tối đa một hành khách tại một thời điểm.*

1.3 Yêu cầu Bưu kiện (Parcel Requests)

Yêu cầu bưu kiện j ($1 \leq j \leq M$) được xác định bởi:

- Điểm nhận hàng: P_j^h;
- Điểm giao hàng: D_j^h;
- Khung thời gian lấy hàng: $[E_j^{h,p}, L_j^{h,p}]$;
- Khung thời gian giao hàng: $[E_j^{h,d}, L_j^{h,d}]$;
- Thời gian dịch vụ lấy và giao hàng: S_j^h;
- Doanh thu: T_j^h;
- Khối lượng bưu kiện: w_j.

Đối với mỗi bưu kiện được phục vụ, việc lấy hàng và giao hàng phải do **cùng một xe taxi** thực hiện, và việc lấy hàng phải diễn ra **trước** khi giao hàng. Gọi $arrP_j^h$ là thời điểm taxi đến P_j^h. Việc lấy hàng hợp lệ khi:

$$arrP_j^h \leq L_j^{h,p} \tag{6}$$

Thời gian bắt đầu lấy hàng là $srvP_j^h = \max\{arrP_j^h, E_j^{h,p}\}$ và hoàn thành lúc $depP_j^h = srvP_j^h + S_j^h$.

Sau đó taxi sẽ di chuyển đến D_j^h. Gọi $arrD_j^h$ là thời điểm taxi đến D_j^h. Dịch vụ giao hàng chỉ hợp lệ khi:

$$arrD_j^h \leq L_j^{h,d} \tag{7}$$

Nếu taxi đến trước thời gian giao hàng sớm nhất ($E_j^{h,d}$), nó phải chờ. Dịch vụ giao hàng bắt đầu tại $srvD_j^h = \max\{arrD_j^h, E_j^{h,d}\}$ và hoàn thành lúc $depD_j^h = srvD_j^h + S_j^h$.

1.4 Sức chứa của Taxi (Taxi Capacity)

Taxi k có sức chứa bưu kiện là Q_k. Khi bưu kiện j được lấy bởi taxi k, tải trọng hiện tại của taxi tăng lên w_j. Khi bưu kiện j được giao, tải trọng giảm đi w_j. Tại mọi thời điểm, tải trọng của xe phải thỏa mãn:

$$load_k \leq Q_k \tag{8}$$

**Sự có mặt của hành khách không đóng góp/tính vào ràng buộc sức chứa bưu kiện này.*

1.5 Lộ trình và Chi phí Di chuyển (Routes and Travel Cost)

Một lộ trình của taxi k là một chuỗi sắp thứ tự các điểm ghé thăm:

$$R_k = (v_0^k, v_1^k, \ldots, v_{m_k}^k) \tag{9}$$

trong đó $v_0^k = O_k$ là vị trí ban đầu của taxi k (sẵn sàng từ thời điểm $dep(v_0^k) = 0$), và mỗi điểm tiếp theo $v_r^k \in \{P_i^n, D_i^n, P_j^h, D_j^h : 1 \leq i \leq N, 1 \leq j \leq M\}$ là điểm đón/trả của một yêu cầu được phục vụ. Taxi không cần quay lại điểm tập kết sau khi kết thúc lộ trình hoặc trong lúc chờ yêu cầu tiếp theo.

Với hai điểm liên tiếp $u = v_{r-1}^k$ và $v = v_r^k$ ($1 \leq r \leq m_k$) trên lộ trình R_k, taxi mất $t(u, v)$ thời gian di chuyển và chịu chi phí $c(u, v)$. Lộ trình phải thỏa mãn tất cả các ràng buộc về thứ tự ưu tiên, chuyến đi trực tiếp của khách, khung thời gian và sức chứa bưu kiện.

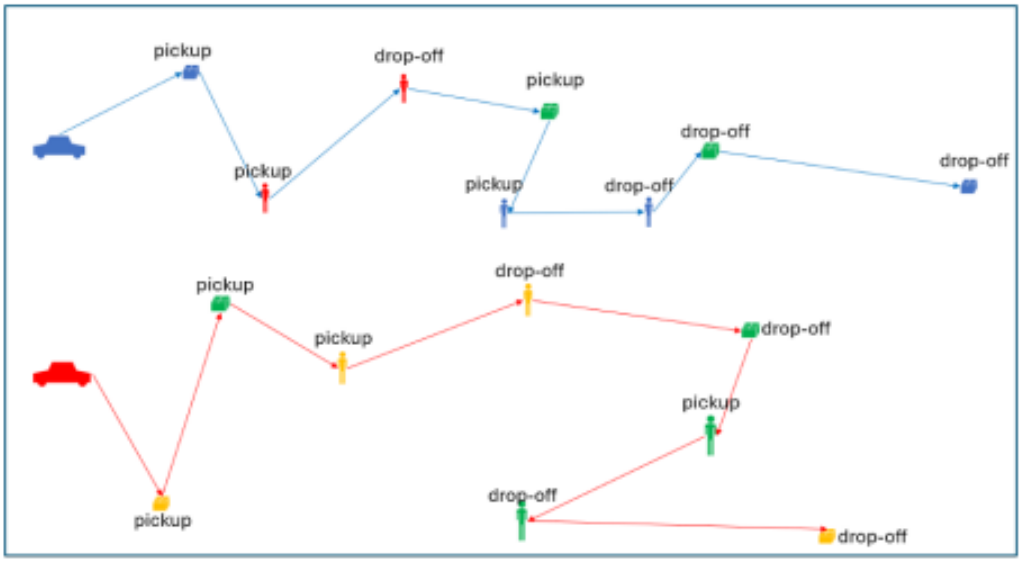
Tổng số nút dịch vụ và điểm xuất phát ban đầu (các đỉnh) trong mạng lưới là $2N+2M+K$.

1.6 Hàm Mục tiêu (Objective Function)

Gọi S_N là tập hợp các yêu cầu hành khách được phục vụ và S_M là tập hợp các yêu cầu bưu kiện được phục vụ. Lợi nhuận (Benefit) của toàn bộ lịch trình được tính theo công thức:

$$\text{Benefit} = \sum_{i \in S_N} T_i^n + \sum_{j \in S_M} T_j^h - \sum_{k=1}^{K} \sum_{r=1}^{m_k} c(v_{r-1}^k, v_r^k) \tag{10}$$

Mục tiêu là tính toán một lịch trình hợp lệ sao cho **tối đa hóa Benefit**. Công ty **không bắt buộc** phải phục vụ tất cả các yêu cầu hành khách hay bưu kiện.



Hình 1: Lấy từ đề bài SOICT 2026

2 Phân Tích Yêu Cầu Bài Toán & Bảng Tóm Tắt

2.1 Tóm tắt Đọc hiểu Bài toán

- **Bản chất toán học:** Bài toán thuộc nhóm **PDVRPTW** (*Pickup and Delivery Vehicle Routing Problem with Time Windows*), mở rộng thêm tính chất kết hợp chở khách + bưu kiện và cho phép bỏ qua yêu cầu nếu không có lời (Selective VRP).
- **Quyết định cần đưa ra trong Code:**
  1. Chọn tập hợp các hành khách $S_N \subseteq \{1..N\}$ và bưu kiện $S_M \subseteq \{1..M\}$ sẽ được phục vụ.
  2. Phân công mỗi yêu cầu chọn lựa cho đúng 1 xe taxi k.
  3. Sắp xếp thứ tự danh sách các điểm ghé thăm R_k cho từng taxi k.

2.2 Bảng Tóm tắt Ràng buộc Cứng (Hard Constraints)

Bảng 1: Bảng tra cứu quy tắc hợp lệ cho lộ trình xe

Đối tượng	**Tên Ràng Buộc**	**Mô tả chi tiết**
Taxi	Xuất phát	Bắt đầu tại O_k lúc $t = 0$. Không bắt buộc quay về O_k.
	Sức chứa Q_k	Tổng khối lượng hàng hiện tại trên xe $\leq Q_k$ tại mọi thời điểm.
Hành khách	Đi trực tiếp	Đón tại $P_i^n \rightarrow$ Trả tại D_i^n trực tiếp. **Cấm** ghé bất kỳ điểm nào khác ở giữa.
	Sức chứa khách	Tối đa 1 hành khách trên xe tại một thời điểm.
	Khung thời gian	Đến P_i^n trước L_i^n. Thời gian đón/trả tốn S_i^n.
Bưu kiện	Nhận trước - Giao sau	Điểm nhận P_j^h phải nằm trước điểm giao D_j^h trong lộ trình của cùng 1 taxi.
	Gom/Giao hàng	Được chở nhiều bưu kiện cùng lúc (miễn là tổng tải trọng $\leq Q_k$).
	Khung thời gian	Đến nhận trước $L_j^{h,p}$, đến giao trước $L_j^{h,d}$. Thời gian bốc dỡ tốn S_j^h.

2.3 Danh mục Các Biến Dữ Liệu Cần Quản Lý Trong Python

- **Cấu trúc Điểm (Node):**
  - $0 \ldots K-1$: Các điểm xuất phát của taxi O_k.
  - Các cặp điểm khách: (P_i^n, D_i^n) kèm $T_i^n, [E_i^n, L_i^n], S_i^n$.
  - Các cặp điểm hàng: (P_j^h, D_j^h) kèm $T_j^h, [E_j^{h,p}, L_j^{h,p}], [E_j^{h,d}, L_j^{h,d}], S_j^h, w_j$.
- **Ma trận:** `time_matrix[u][v]` lưu $t(u, v)$ và `cost_matrix[u][v]` lưu $c(u, v)$.

3 Kế Hoạch Thực Hiện & Chiến Thuật Thuật Toán

3.1 Định Hướng Chiến Thuật Thuật Toán

- **Hướng chính (Primary Strategy):** Thuật toán **ALNS** (Adaptive Large Neighborhood Search). Thuật toán hoạt động bằng cách phá hủy một phần lộ trình (Destroy Operators) và xây dựng lại (Repair Operators) dựa trên trọng số điểm thưởng.
- **Hướng dự phòng (Backup Strategy):** **Greedy Insertion** kết hợp **Simulated Annealing (SA)** hoặc sử dụng bộ giải **Google OR-Tools** nếu ban tổ chức cho phép dùng thư viện ngoài.

3.2 Mốc Thời Gian Quan Trọng (Timeline Key Dates)

- **Đạn đăng ký (Registration Due):** 15/09/2026.
- **Giai đoạn Thử nghiệm (Probation Period):** 20/09/2026 – 20/10/2026.
- **Vòng Chung kết (Main Competition):** 01/11/2026 – 04/11/2026.

3.3 Lộ Trình Chi Tiết Từng Tuần (Giai Đoạn Probation)

3.3.1 Tuần 1 (20/09 – 27/09): Nền Tảng Data Structure & Lời Giải Cơ Sở

- **Nhiệm vụ chính:**
  - Xây dựng các lớp/cấu trúc dữ liệu trong Python (Node, Request, Route, State).
  - Viết hàm kiểm tra tính hợp lệ của lộ trình (Checker/Validator).
  - Hiện thực thuật toán **Greedy Insertion** đơn giản để tạo ra lời giải ban đầu hợp lệ ($Benefit > 0$).
- **Phương án Backup (Nếu bị kẹt):** Nếu các ràng buộc về thời gian làm thuật toán Greedy không chèn được đơn nào, chuyển sang **Random Insertion** kèm cơ chế bỏ qua đơn vi phạm để đảm bảo có kết quả nộp lấy điểm sàn.

3.3.2 Tuần 2 (28/09 – 04/10): Tối Ưu Cục Bộ (Local Search)

- **Nhiệm vụ chính:**
  - Phát triển các toán tử di chuyển điểm: `Relocate`, `Swap`, `2-opt` thích ứng cho việc ghép hành khách/hàng hóa.
  - Kết hợp các toán tử với thuật toán Hill Climbing để cải thiện $Benefit$.
- **Phương án Backup (Nếu bị kẹt):** Nếu việc Swap làm hỏng tính chất "chở khách đi thẳng", hãy tách riêng 2 tập toán tử: một tập dành riêng cho tuyến khách và một tập dành riêng cho tuyến bưu kiện.

3.3.3 Tuần 3 (05/10 – 12/10): Tích Hợp ALNS

- **Nhiệm vụ chính:**
  - Xây dựng các hàm phá hủy (Random Destroy, Worst Cost Destroy, Shaw Destroy).
  - Xây dựng các hàm sửa chữa (Greedy Repair, Regret Insertion).
  - Cập nhật trọng số toán tử động dựa trên điểm số thu được qua mỗi vòng lặp.
- **Phương án Backup (Nếu bị kẹt):** Nếu ALNS sinh quá nhiều lỗi phát sinh hoặc không hội tụ, lập tức chốt phương án dự phòng: dùng **Simulated Annealing** chạy trên nền Local Search của Tuần 2.

3.3.4 Tuần 4 (13/10 – 20/10): Tinh Chỉnh Tham Số & Tối Ưu Tốc Độ

- **Nhiệm vụ chính:**
  - Tinh chỉnh các tham số (nhiệt độ giảm trong SA, hệ số thưởng trong ALNS).
  - Tối ưu hóa hiệu năng code Python (sử dụng PyPy, vectorized numpy, hoặc cache kết quả tính toán khoảng cách/thời gian).
- **Phương án Backup (Chống TLE):** Thiết lập cơ chế ngắt thời gian cứng (`timeout guard`). Nếu gần chạm giới hạn thời gian (Time Limit), chương trình sẽ dừng vòng lặp ngay lập tức và in ra kết quả tốt nhất đạt được.

3.4 Giai Đoạn Vòng Chung Kết (01/11/2026 – 04/11/2026)

- **Nhiệm vụ chính:** Kiểm thử trên các testcase đặc biệt (edge cases: toàn bộ là đơn bưu kiện, hoặc mật độ khách quá dày đặc).
- **Nguyên tắc:** Cố định cấu trúc thuật toán cốt lõi, không sửa logic lớn trong giai đoạn này để tránh phát sinh lỗi không mong muốn.